Measuring Trust Concentration as an Architectural Property

June 2026
Revision 2.1
Zach Miller (NoNo)

Abstract

Modern organizations rely on systems that accumulate authority by design. Identity providers, endpoint management platforms, CI/CD pipelines, DNS, package registries, and secrets managers all sit in this group. They are coordination layers. They are trusted to act across the whole estate because that is their job.

When one of these systems is compromised, the result is often rapid, organization-wide impact. This happens even when the system is operating exactly as intended. No exploit is required. The system does what it was built to do, for the wrong person.

This kind of risk is largely invisible to the frameworks organizations already use. A company can hold SOC2, ISO, CMMC, EDR, MFA, and a SIEM, and still operate a single control plane capable of destroying everything. None of those frameworks measure that. The gap is not that the danger is unknown. The gap is that trust concentration is not treated as a thing you measure, compare, or track.

This work argues that trust concentration deserves to be a first-class architectural risk. This work presents a way to measure it directly. It does so by measuring delegated authority, the legitimate enterprise capability entrusted to a system, and then reading the pattern of delegated authority across an environment as authority concentration. The model was stress-tested against its own indicators and got smaller under testing, not bigger: it started at fourteen indicators and collapsed to seven once correlation analysis showed several were measuring the same thing. The model works in two steps. First it classifies the authority delegated to a system by design, the legitimate enterprise capability the system has been entrusted with. Then it measures two consequences of that capability: how much damage its abuse could cause, which this work calls Structural Blast Radius, and how hard that abuse is to see, which this work calls Opacity. Capability is the input. Blast radius and opacity are what follow from it. The contribution is not the discovery that these systems are dangerous. Experienced architects already know that. The contribution is to propose that authority concentration is an architectural property worth measuring in its own right, alongside the properties security practice already tracks, and to give organizations a shared way to reason about it. The seven indicators are a first attempt at operationalizing that idea, not the idea itself.

1. The gap

Most of security is about making trust smaller. Least privilege. Zero Trust. Attack surface reduction. All of it tries to reduce how much trust exists so there is less to lose.

That work is necessary. It is not enough.

Trust is never zero. Modern enterprises deliberately build trust concentration because it is economically efficient. Centralized identity, orchestration, endpoint management, and deployment systems reduce operational cost precisely by concentrating authority. The same architectural property that makes these systems valuable also makes them uniquely dangerous when compromised. This framework does not argue that such systems should not exist. It argues that their concentration of authority should be treated as a measurable architectural property rather than an implicit assumption. Some systems have to be trusted to do their job. So, there is a question security practice has historically set aside. What happens when one of those trusted systems is abused? Not how to prevent it. What capability has been concentrated there, and what happens when it is turned against you.

We have good tools for prevention. We have limited shared language for the failure itself.

One thing to be clear about up front. This framework intentionally ignores likelihood. Security people are trained to think of risk as likelihood times impact. This model measures a form of impact only: what a system can do when its trust fails, not how probable that failure is. Likelihood is a separate question, and a real one, but it is not what this aims to measure.

Here is a sharper version of the gap. A company can pass every audit it is required to pass. SOC2. ISO 27001. CMMC. It can run EDR on every endpoint, enforce MFA everywhere, and feed a SIEM. It can still have one control plane that, if compromised, has the potential to end the company.

The danger is not hidden because it is subtle. It is hidden because nothing is pointed at it. Organizations routinely inventory assets, vulnerabilities, identities, and controls. How often do they inventory concentrations of authority?

The example

A monitoring tool changes its dashboard background after a config push. The dashboard looks different on Monday. That is a change to a trusted system.

An attacker takes over your domain controller and makes unnoticed changes. That is also a change to a trusted system.

Your tooling sees both the same way: an authorized change happened. The logs look alike. The alerts, if any fire, look alike.

These are not the same event. One is a Tuesday. The other is the company going into crisis mode. Both were authorized. Both were logged. The difference is structural: what each system can do when its trust is turned against you, and how hard that use is to see.

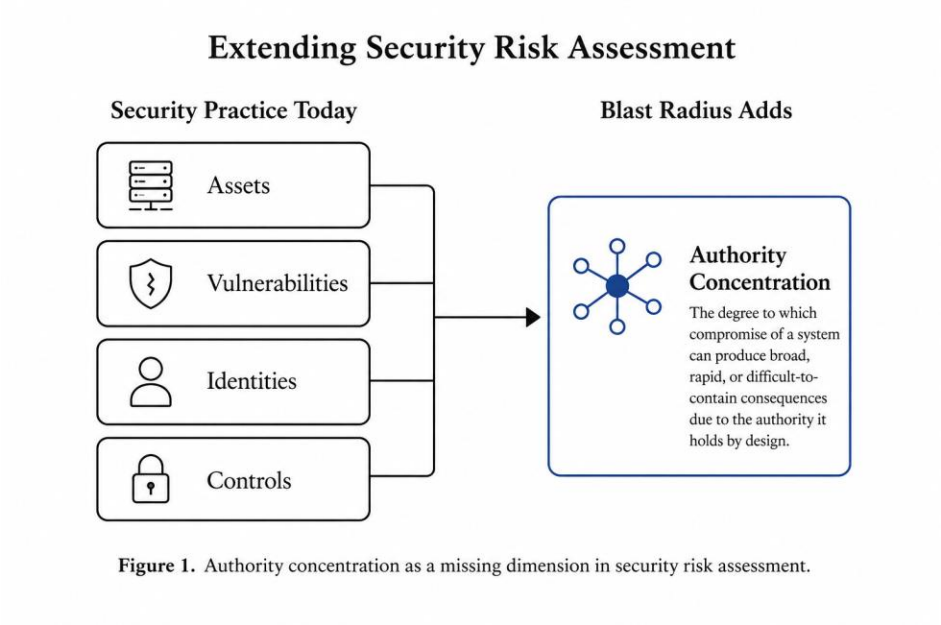

Figure 1. Authority concentration as a missing dimension in security risk assessment.

2. Delegated authority

Before the model can measure anything, it needs a precise name for the thing it measures. That thing is delegated authority.

Delegated authority is the legitimate enterprise capability entrusted to a system so that it can do its job. Active Directory has been entrusted to manage identities. A configuration manager has been entrusted to run code on its fleet. An orchestration platform has been entrusted to execute administrative actions across hosts. The question delegated authority asks is simple: what has this system been entrusted to do?

This is deliberately not a question about vulnerability, privilege level in the operating-system sense, or the quality of a deployment. A system can have no known vulnerabilities, be configured to every best practice, and still hold enormous delegated authority, because the authority is the point of the system. Delegated authority is what the enterprise handed the system on purpose. The framework measures that.

Two clarifications matter here, because they are easy to get wrong. Both turn on a distinction worth naming up front. There are two layers to what is being measured. The first is the technology class: what kind of authority can a system of this type hold at all. An execution platform can run code on hosts; an identity provider can assert identity. That class is fixed regardless of how any one organization deploys it. The second is the deployment realization: how much of that authority a particular instance has actually been delegated, including how far it reaches. The framework uses both. It classifies the technology class and scores the realized authority of the deployment in front of it. The two clarifications below each sit on one of these layers.

First, reach is part of delegated authority. A system entrusted to act on fifty thousand endpoints has been delegated more authority than the same product entrusted to act on five. This is why the instrument includes a reach indicator and scores the realized authority of a deployment, not just the

abstract product category. The technology class may be the same across two deployments of the same tool, but the authority actually delegated is not identical when the breadth of targets differs. The framework scores that realized authority. What it intentionally does not do is treat organization size, business importance, or deployment scale as a proxy for danger in its own right. Reach is an input to delegated authority, not a popularity contest between companies.

Second, the consequence of abuse scales with deployment even when the class of authority does not. Ansible managing five systems and Ansible managing fifty thousand are the same kind of tool with the same kind of delegated authority. The organizational consequence if either is abused is very different. The framework keeps these separate on purpose. Delegated authority describes what the system can do. Organizational consequence describes how much it would hurt. Reach connects them, which is why reach is measured and consequence is acknowledged rather than scored as if it were a property of the technology itself.

> The framework works at two layers. It classifies the technology class, the kind of authority a system can hold, and it scores the realized authority of the specific deployment, including reach. The same orchestration platform managing ten servers and managing ten thousand shares one technology class but has been delegated different realized authority. What the framework never does is treat business size by itself as the measure of danger.

3. Delegated authority versus authority concentration

Delegated authority is a property of one system. Authority concentration is a property of an architecture. Keeping these at different levels is what lets the model say something a single score cannot.

When you score one system, you learn how much authority has been delegated to it. That is useful, and it is what every individual score in this work represents. But the more important pattern only appears when you score many systems in the same environment. An enterprise that has delegated enormous authority to Active Directory, an endpoint manager, a configuration manager, an orchestration platform, a certificate authority, and a secrets manager has not made six independent decisions. It has concentrated authority into a handful of systems. That concentration is an architectural fact about the enterprise, not about any one tool.

This is the difference between describing a technology and describing a design. Saying Active Directory holds broad delegated authority is a statement about a technology. Saying an enterprise has concentrated its operational authority into six systems, any one of which would cascade across business units if compromised, is a statement about that enterprise's architecture. The first is what an individual score captures. The second is what the matrix reveals when individual scores are placed side by side.

This is why the matrix is not merely a visualization. The individual scores measure delegated authority. The pattern of those scores across an environment exposes authority concentration, which is the architectural property that explains why a single compromise so often cascades through an organization that believed its systems were independent. The instrument operates at the level of delegated authority. Concentration is what emerges from it.

The relationship between the two can be stated as a short chain.

1. A technology possesses delegated authority, the legitimate capability it has been entrusted with.
2. The framework classifies that delegated authority using the indicators described later.
3. When many high-authority technologies coexist in the same environment, that architecture exhibits authority concentration.
4. The matrix exists to reveal that concentration, by placing individual scores side by side so the pattern becomes visible.

This gives every term in the framework a distinct role, which is worth stating plainly because trust, authority, and concentration are easy to blur together:

- **Delegated authority** is a property of a technology. It is what one system has been entrusted to do.
- **Authority concentration** is a property of an architecture. It is what emerges when many high-authority technologies coexist.
- **Structural Blast Radius** is the consequence of concentrated delegated authority. It is how much damage abuse could cause.
- **Opacity** is the visibility of misuse. It is how hard that abuse is to distinguish, attribute, and reconstruct.

Keeping these four terms distinct is deliberate. Each one answers a different question, and collapsing any two of them back together is how this kind of model usually loses precision.

4. Reading concentration across an environment

A natural question follows from the distinction between delegated authority and authority concentration. If concentration is an architectural property, when is there too much of it? The honest answer is that the framework does not say, and that refusal is deliberate.

The framework reveals authority concentration. It does not prescribe how much concentration is appropriate, because the right level depends on an organization's size, risk tolerance, and operational needs. Inventing a numeric threshold, some rule that five high-authority systems means an environment is dangerous, would be exactly the kind of false precision this work avoids everywhere else. There is no evidence behind such a number, and a reviewer would be right to ask where it came from. What the framework can do is make concentration visible enough to be a deliberate choice rather than an accident.

It helps to think of concentration the way a clinician thinks of blood pressure. A reading of one hundred forty over ninety does not declare that a heart attack is coming. It is a risk factor worth attention. A high concentration of delegated authority is the same kind of signal. It describes a characteristic of the architecture. It does not pronounce a verdict on it.

Shape, not grade

Environments differ not only in how much authority they concentrate but in how that authority is distributed. That shape can be described without being ranked.

Some environments are centralized, with a few systems holding extremely high authority. Some are distributed, with authority spread across many systems of moderate authority and few extreme ones.

Some are identity-centric, with authority pooled in identity infrastructure. Some are operations-centric, with authority pooled in deployment and orchestration. Some are mixed, with high authority spread across several domains at once.

These are descriptions, not grades. A distributed topology is not inherently safer than a centralized one. They fail differently. A distributed environment may have more independent points of failure but a harder recovery story when several are touched at once. A centralized one concentrates risk into fewer systems but may be easier to monitor, partition, and reason about. The framework does not claim to know which shape is better for a given organization. It claims only that the shape is worth seeing clearly.

A discussion trigger, not a cutoff

Because there is no defensible threshold, the framework replaces the idea of a cutoff with the idea of a discussion trigger. Where many systems in an environment carry high delegated authority, that pattern is worth a deliberate architectural review. Not because concentration is wrong, but to confirm it was intended, to check whether genuinely independent control planes exist, and to ask whether additional segmentation or separation of duties would reduce systemic risk at an acceptable cost.

This is how mature risk disciplines tend to work. They do not declare that an architecture has failed. They highlight characteristics that merit deeper analysis. The useful question this framework provokes is not "did we pass" but "did we mean to concentrate this much authority here, and are we comfortable with what happens if one of these systems is turned against us."

5. Two axes

The model separates two things most frameworks blend together.

Structural Blast Radius

The degree to which compromise of a system can produce broad, rapid, or difficult-to-contain consequences due to the authority the system possesses by design.

The key words are "by design." This is not about misconfiguration. A domain controller that passes every audit is still a domain controller. The danger survives a clean review. The goal here is to measure what the system can do when it is working correctly, and the wrong person is driving.

This is also what separates the model from simply ranking how much power a system has. One of the five indicators inside this axis is Irreducible Authority. It asks how much of a system's authority is unavoidable even in a perfectly clean, fully engineered deployment. Some authority is optional and can be scoped away. Some is architecturally required for the system to do its job at all. An identity provider must be trusted to assert identity. A configuration manager must be trusted to run code on its fleet. You cannot configure that away without breaking the product. That is the danger that survives a clean audit, and it is the part existing severity and tiering models do not capture. Two systems can hold the same raw power, and the one whose power is irreducible is the harder problem.

This is also why the model does not score configuration, compliance, or likelihood. Consider two deployments of the same orchestration platform. One uses MFA, separation of duties, change approvals, and full logging. The other uses none of them. The first is clearly more secure. But both deployments hold the same underlying authority to change systems at scale. Security controls reduce the likelihood that authority will be abused. They do not reduce the amount of authority present. This model measures the structural consequences of that authority, not the quality of its implementation.

Opacity

The degree to which malicious use of a system's authority is difficult for defenders to distinguish, attribute, or reconstruct.

When the dashboard changes, you notice. When an attacker uses domain admin to issue a valid-looking ticket, it looks exactly like normal work. There is no malware. There is no broken rule. The action is signed, authorized, and policy-valid. Opacity is how hard it is to tell that apart from legitimate use, and how hard it is to figure out who did it after.

Why two and not one

These answer different questions, and the difference is not academic. Blast radius is how much damage a system can do. Opacity is whether you will see it happen and how long you have to respond. The amount of authority a system holds does not change based on how well it is logged. A control plane that can reach thirty thousand endpoints has that reach whether its actions are fully audited or completely silent. Logging does not shrink the blast radius. It changes whether the blast is something you can catch in progress or something you reconstruct afterward, if at all.

That is why opacity is not a second measure of danger. It is a modifier on the danger blast radius already describes. Two systems with the same blast radius are not the same risk if one is observable and the other is not. This is also why collapsing the two into a single score would lose the information that matters most. A single number cannot tell you whether to invest in partitioning or in telemetry. The two axes kept apart can.

6. Relationship to Zero Trust and variable trust

Some readers will naturally compare this model to Zero Trust. The two are complementary, not alternatives. Zero Trust, variable trust, NIST, ISO 27001, and CMMC operate earlier in the lifecycle of trust; this model operates after authority has been delegated.

Zero Trust, least privilege, NIST control families, ISO 27001, CMMC, and the broader family of variable-trust and compliance approaches are concerned with establishing, adapting, and protecting trust. They ask who and what should be trusted, under what conditions, and how that trust should be continuously verified and constrained. Those are decisions about whether to grant authority and how tightly to govern its use.

This model begins after those decisions have been made. It assumes authority has already been delegated, because in every real enterprise some authority always is, and asks a different question: what enterprise capability has been delegated to those systems, and what pattern of authority concentration emerges across the architecture if one of them is turned against you. Zero Trust works to reduce the likelihood that delegated authority is abused. This work classifies that delegated authority and measures the consequence if those protections fail, revealing where authority concentration exists.

That makes the two complementary rather than competing. A mature environment still needs Zero Trust to govern access and least privilege to keep delegation minimal. It also needs a way to see where, despite all of that, authority has unavoidably pooled. This model is meant to fill that second role, not replace the first.

7. The quadrant

Stack the two axes and you get four kinds of systems. This is the part a practitioner actually uses.

	Low Opacity (visible)	High Opacity (blind)
High Blast Radius	Scary but governable. Add approvals, partitioning, recovery paths.	Highest-priority architectural concern. Can rewrite reality before defenders observe it. Telemetry first, then partitioning.
Low Blast Radius	Boring. Baseline control.	Annoying, not existential. Do not over-invest.

The top-right cell is the one that matters most. High blast radius and high opacity. The system can do enormous damage and you will not know until it is done. Endpoint management platforms and identity providers commonly fall in this quadrant, not as a claim about any specific product, but because that class of system holds broad authority by design and its legitimate use is hard to tell from abuse.

This is the practical payoff. The purpose of the model is not to find vulnerabilities. It is to find where architectural investment produces the greatest reduction in systemic risk. A high-blast, high-opacity system is a different kind of problem than the vulnerability of the week. It needs a different kind of response, and the quadrant tells you which. Different cells get different investment. That is a decision the model makes for you that a vulnerability count cannot.

8. What this is for

A seasoned architect does not need this model to know that Okta, SCCM, Terraform org admin, and a DNS root zone are dangerous. They will look at a high score and think, yes, obviously. They are not wrong.

The model does not exist to teach experts what they already know. It exists to give organizations a shared way to reason about a risk that their existing frameworks do not capture.

The closest familiar comparison is CVSS. This work is not a replacement for vulnerability scoring systems such as CVSS, and it is not "CVSS for trust." CVSS provides a common language for discussing the technical severity of vulnerabilities. It asks what happens if this weakness is exploited. This model asks a different question. What happens if this trusted authority is compromised. A system may have no known vulnerabilities, pass every audit, and still represent a major source of systemic risk because of the authority it holds by design. That is the category this model addresses.

CVSS is useful for more than its score. Much of its value is the social function. It gave people a shared landmark so a severity conversation could happen at all. This model is trying to do the same thing for trust concentration. Consider a design review. You say "Jenkins has too much authority." The honest reply is "compared to what." Without a shared way to express it, the claim is a judgment call, and judgment calls lose to whoever owns the system. Now you say "Jenkins scores high on structural blast radius: broad authority, low partitionability, and it acts on its own." Someone can still disagree. But now they have to say where. Is the authority not that broad. Can it be partitioned after all. The disagreement

moves from whether your concern is legitimate to which specific input is wrong, and that is a question evidence can settle.

There is a reaction this work invites from experienced practitioners: "this is cute, I already know SCCM is dangerous." That reaction is correct, and it misses the point. Being able to recognize a dangerous system was never the gap. The gap is everything that comes after recognition.

The cost of that gap is not hypothetical. "I already know it's dangerous" is a comfortable position right up until the system in question is the one that fails, and then the same knowledge that felt obvious becomes a question no one can answer: if everyone knew, why are we here? Recognition that is never written down, compared, or reported is not protection. It is intuition that lives in a few people's heads and leaves when they do. The purpose of this work is to move that intuition into something an organization can hold, rank, and act on, rather than something it rediscovers after an incident.

9. Intentional exclusions

It is worth stating plainly what this framework does not evaluate, because several of these are the first things a security reader expects to see.

The framework does not score security controls, MFA, privileged access management, NIST control families, ISO 27001, CMMC, vulnerability state, or organizational maturity.

This is not an oversight. Every item on that list influences the likelihood that delegated authority is abused. None of them change the amount of authority delegated to the system. A configuration manager that can run code on every endpoint can still do so whether or not the organization holds a certification, enforces MFA on the console, or runs a mature PAM program. Those controls make abuse less likely and are worth every investment. They do not shrink the delegated authority, and delegated authority is what this framework measures. Mixing the two would reintroduce exactly the confusion the model is trying to remove: it would let a well-run deployment of a catastrophic system score as though it were safe, when what has actually happened is that abuse has been made less likely, not less consequential.

10. The indicators

The two axes are not scored directly. They are made of smaller, concrete indicators that a person can actually evaluate. Raters score the indicators. The axis position is derived from them.

Structural Blast Radius is built from five indicators. Authority Magnitude. Irreducible Authority. Total Plane Reach. Native Partitionability. Autonomous Actuation.

Opacity is built from two. Operational Discriminability. Provenance Gap.

The five Structural Blast Radius indicators:

- **Authority Magnitude.** Native action capability, taking into account both how deep the privilege runs and how far a single action reaches.
- **Irreducible Authority.** How much of that authority is unavoidable even in an ideal, fully engineered deployment. The structural minimum, not the configured maximum.

- **Total Plane Reach.** The aggregate breadth of targets the system governs. Reach without authority is the low-danger case, so this is scored separately from authority.
- **Native Partitionability.** Whether the normal, recommended operating mode forces authority to collapse into one principal, permits compartmentalization, or enforces partition. Scored as the system is actually run, not as it could theoretically be configured.
- **Autonomous Actuation.** Whether the system acts on its own, on a schedule or in response to events, so that an attacker can ride existing automation rather than trigger anything.

The two Opacity indicators:

- **Operational Discriminability.** How indistinguishable a malicious authorized action is from legitimate operation, including whether a stable baseline of normal even exists. Higher means harder for a defender.
- **Provenance Gap.** How far telemetry fails to attribute the actor or origin behind an action. Higher means harder for a defender.

Full anchors and scoring guidance live in the instrument.

One indicator, Recovery Dependency, is carried as provisional. It asks whether the system is also the path to recover from its own compromise. It is scored but not yet assigned to an axis.

11. How the indicators were chosen

This is the part that separates a measurement from an opinion.

Every indicator has to pass two tests to stay. This is called the constitution.

First, the independent lever test. Can the property be improved without materially changing something the instrument already scores. If the only way to fix it is to change another indicator, it is not its own thing. It is a symptom.

Second, the architectural property test. Is it a structural fact about the system, not an organizational or maturity fact. This blocks things like training and runbooks. Those matter, but they are not properties of the system.

An indicator has to pass both. Passing one is not enough.

The graveyard

This started with fourteen indicators. I did not keep them because I liked them. They were tested, and I cut the ones that failed.

Indicator	Disposition	Reason
Privilege Depth	Folded into Authority Magnitude	Correlated near 0.98 with Fan-out. The signal is preserved as an input to Authority Magnitude rather than a separate column.

Single-action Fan-out	Folded into Authority Magnitude	Correlated near 0.98 with Privilege Depth. Preserved as the reach dimension of the Authority Magnitude anchor.
Propagation Latency	Folded into the derived output	Correlated near 0.90 with Fan-out. Feeds the derived Defender Gap, not a standalone column.
In-flight Reversal Window	Folded into the derived output	Correlated near 0.90 with Defender Gap. An input to the derived output, not its own dimension.
Defender Gap	Removed as a scored column, kept as a derived output	Did not satisfy the constitution under current evidence. Every way to improve it routed back to authority or telemetry. Correlated 0.91 with Authority Magnitude. Reopenable if an independent lever is ever found.

This started with fourteen indicators and kept seven, plus one derived output and one provisional column.

I decided to show the dead indicators on purpose. It answers a fair criticism before it is made. Did you remove things when the data disagreed with your intuition. Yes. Here is the list.

12. Testing our own work

A model that only makes sense to the people who built it is not validated. It is just coherent.

I ran a correlation analysis on the early scores with the help of some peers. The result was uncomfortable and useful. One factor explained most of the variance. Fourteen indicators were measuring about three or four things. That is false precision. There were columns that looked independent and were not. These figures come from a small, early, and deliberately selected sample. They are indicative, not confirmatory, and they are reported here to show the direction the evidence pushed the model, not as a validated result.

So, they were collapsed. Two indicators that correlated at 0.98 became one. A modifier that correlated with authority at 0.91 was removed as a scored column and turned into a derived output. The model went from fourteen columns to seven plus one derived output. It got smaller under testing, not bigger.

To be specific about the numbers. In the first analysis, one factor explained about 73 percent of the variance across the indicators, and roughly four components covered 90 percent. Fourteen columns were carrying three to four real dimensions. Several pairs were nearly collinear: Privilege Depth and Fan-out at 0.98, and Defender Gap with Authority Magnitude at 0.91. I then scored a wider, deliberately

varied set of systems to spread the range, and the strongest collinearity relaxed, which is what separated the genuinely independent indicators from the redundant ones. The independent survivors, including Provenance Gap, Autonomous Actuation, Total Plane Reach, and Partitionability, held their own signal across both rounds.

There has been a deliberate effort to avoid claiming a precise danger formula. The interaction between the two axes is real, but whether it is additive, multiplicative, or something else is an open question. I leave the exact equation to a larger, subsequent study rather than invent one I cannot defend this early on.

13. Validation

At the time of this paper's publishing, further validation is taking place. An independent scoring trial is underway in which raters who did not build the model score a set of systems from reputation-stripped descriptions. Results will be reported separately.

14. The pattern in the wild

Look at the last few years.

SolarWinds. Kaseya. Okta. CircleCI. Stryker.

This model was partly motivated by the observation that incidents such as these appear to share a structural pattern. I have tended to file them under supply chain. That label may be hiding what they have in common. In each one, a trusted coordination layer was the weapon. The thing trusted to run everything got turned, and it ran everything for the attacker instead. Getting to that layer sometimes took an exploit. Using it did not. Legitimate functionality. Trusted vendor pipeline. Broad reach.

If that hypothesis holds, these are not a string of unrelated incidents. They are the same shape repeating. A trusted plane, high blast radius, high opacity, used exactly as designed against the people who deployed it. Testing whether that shape is real and measurable is part of what this model is for. I state it as a hypothesis on purpose. The model is the thing meant to test it, not the proof that it is true.

The SCCM attack chain studied in the management-plane work referenced below is a concrete case in the high blast radius, high opacity quadrant. The system holds broad authority by design. A single network access account credential extracted from it exists on every managed endpoint, and a relay path leads to full administrative control of the site hierarchy. That is the blast radius. The opacity was measured directly. Logs from the full attack chain were evaluated against the complete community Sigma ruleset, 2,886 rules in total. Of eleven distinct attack steps, only four produced any detection, and all four were generic patterns unrelated to SCCM. The seven steps specific to the management plane were invisible. After writing six custom rules, coverage of the steps that produce evidence in default logging rose from forty-four percent to seventy-eight percent. The remaining gap was a log source problem, not a rule problem: the relevant events are not recorded in default Windows logging or required Sysmon. This is a concrete example of what high opacity looks like in practice. The authority operates as designed, the abuse is indistinguishable from legitimate use, and the telemetry needed to tell them apart is off by default.

15. Threats to validity

This is early work and it has real limits. Naming them is part of the method.

Single-rater testing influenced early development. Much of the indicator refinement happened against a few raters' scores before independent scoring began. That is a source of bias and it shaped the model.

System selection was intentional, not random. The systems scored so far were chosen to stress the model, including deliberate edge cases. That is useful for development and it is not a representative sample.

Validation has not been completed. As of this draft, the instrument is frozen for a first validation round that has not yet run. Claims of external validity are not yet earned.

The model may not generalize past enterprise control planes. It was built from management-plane and identity examples. Whether the indicators behave sensibly on very different kinds of systems is an open question.

There is also an honest tension worth stating. A seasoned architect may not need this model personally. They can already see the danger. The argument is that organizations need it collectively, the way risk disciplines like aviation and reliability engineering use shared language so that concerns become discussable and trackable. That is a claim about organizational value, not personal insight, and it should be read that way.

16. Concentration analysis in practice

The framework is built to do more than score systems one at a time. Once a set of systems has been scored, the more useful question is architectural: where has the delegated authority pooled? Scoring is the input. Concentration is what the scores reveal when read together.

The companion workbook demonstrates this directly. It sets aside the low-authority systems, keeps the highest delegated-authority ones, and groups them by domain, for example identity, endpoint management, deployment, and networking. The output is not a number. It is a shape. It shows whether an environment's authority is concentrated almost entirely in one domain or spread across several. The finding it surfaces is the architectural one: authority is rarely distributed evenly. It clusters into a small number of enterprise control planes, and those clusters are where architectural review may offer the greatest opportunity to reduce concentration through segmentation or independent control planes.

This view is deliberately descriptive. It does not assign an environment an overall score, and it does not declare a correct amount of concentration. It makes the pattern visible so that an architect can ask whether the concentration was intended. The domain groupings are a lens for seeing where authority clusters, not a fixed taxonomy. An organization may classify its own technologies differently. What matters is that the clustering becomes visible at all.

A note on composed authority

There is a further question this naturally raises. When several high-authority systems cluster in one view, are they actually independent, or do they share an upstream authority they all depend on? An endpoint manager, a deployment pipeline, and a monitoring agent may each be scored separately and yet all depend on the same identity provider. In that case the apparent independence is an illusion, and the real concentration is upstream.

This is the idea of composed authority: authority that propagates across trusted systems through legitimate delegation rather than through direct compromise of a final target. Each system on such a path sees only a legitimate request. The malicious intent exists in the composition, not in any single node, which is why scoring systems in isolation does not surface it. Incidents such as SolarWinds and CircleCI illustrate that authority frequently propagates through chains of trusted systems rather than terminating at one platform.

It is not yet clear whether composed authority is best understood through scoring or through architectural visualization. That is the real open question, and it is genuinely open. It is possible the right treatment is to draw and discuss these dependencies rather than assign them numbers, and it is possible the single-system view, read for concentration as above, already captures most of what an architect needs. The current instrument deliberately stops at the level of the individual system and the patterns that emerge across many of them. Whether composed authority warrants its own measurement is left open, on purpose, rather than half-built here.

17. What this is and is not

This is a way to measure a kind of risk that compliance, control, and vulnerability frameworks do not capture. It tells you which of your trusted systems would end you if they broke, and whether you would see it happening.

It is not a vulnerability scanner. It does not find bugs. It measures structure.

It is not a replacement for least privilege or Zero Trust. Those reduce trust. This measures what is left after you have reduced all you can.

Put most simply: the framework does not tell an organization whether it is secure. It helps explain where it has chosen to centralize enterprise capability.

This work is not finished. The core claim is simple. Some trust has to exist. So the real question is not only how do we trust less. It is which of our trusted systems would level us if they broke, and would we even see it. The larger proposal underneath that question is that authority concentration deserves to be measured as an architectural property in its own right. This work is a first attempt at that measurement, not the last word on it.

Every organization has systems it cannot live without and cannot fully see. Most can name them. Few can measure them, rank them against each other, or show whether the risk is getting better or worse. That gap between knowing and measuring is where this work lives. The danger was never that the risk is unknown. It is that it is untracked.

Citations

1. Common Vulnerability Scoring System (CVSS), version 4.0 Specification. Forum of Incident Response and Security Teams (FIRST). https://www.first.org/cvss/
2. Kindervag, J. No More Chewy Centers: Introducing the Zero Trust Model of Information Security. Forrester Research, 2010.
3. Rose, S., Borchert, O., Mitchell, S., and Connelly, S. NIST Special Publication 800-207: Zero Trust Architecture. National Institute of Standards and Technology, August 2020. https://csrc.nist.gov/pubs/sp/800/207/final

4. Miller, Z. LOLMgmt: Living Off the Land in the Management Plane. NolaCon 2026. https://github.com/ridgestoneco/LOLmgmt_NolaCon2026
5. Cybersecurity and Infrastructure Security Agency. Kaseya VSA Supply-Chain Ransomware Attack. CISA Current Activity, July 2, 2021. https://www.cisa.gov/news-events/alerts/2021/07/02/kaseya-vsa-supply-chain-ransomware-attack
6. CircleCI. Incident Report for January 4, 2023 Security Incident. January 13, 2023. https://circleci.com/blog/jan-4-2023-incident-report/
7. Okta. Unauthorized Access to Okta's Support Case Management System. Okta Security, October 2023. https://sec.okta.com/articles/2023/10/unauthorized-access-oktas-support-case-management-system/
8. Microsoft Security Response Center. Microsoft Internal Solorigate Investigation Final Update. February 18, 2021. https://msrc.microsoft.com/blog/2021/02/microsoft-internal-solorigate-investigation-final-update/
9. Microsoft. Securing Privileged Access: Enterprise Access Model. Microsoft Learn. https://learn.microsoft.com/en-us/security/privileged-access-workstations/privileged-access-access-model
10. SpecterOps. Misconfiguration Manager: Overlooked and Overprivileged. June 2025. https://specterops.io/blog/2025/06/26/misconfiguration-manager-still-overlooked-still-overprivileged/
11. International Organization for Standardization (ISO) and International Electrotechnical Commission (IEC). ISO/IEC 27001:2022 - Information security, cybersecurity and privacy protection - Information security management systems - Requirements. Geneva, Switzerland: ISO, 2022.